Dose calculation Project

Students: Max ... ...

Lisa Rallo lisa.rallo@student.uclouvain.be

In this project, we began by initializing several constants essential for our calculations, including the masses of all particles (in units of MeV/c^2), the speed of light, the minimum energy transfer, the constant parameter E_s, the radiation length of water, the density of water, and the number of electrons per atom.

I.A Electromagnetic interactions

To compute the CSDA, we created the function **CSDA()**, which takes the following parameters as input, referring to the particle to be initialized: *energy, direction, position, and initialize*. The parameters *energy, direction, and position* are set in default to 0 (but will get real values afterwards), and *initialize* is set to **True in default**. These parameters will be particularly useful for implementing the I.B. part of the task.

If the input ***initialize* == True**, the function initializes a new particle, where its position in the transversal plane is given by the beam position plus a sampling from a Gaussian distribution, with the standard deviation stored in the **Spot_size** array, while along the z-axis, the position remains the same as the beam's position.

The direction of the particle corresponds to the direction of the beam, while the energy of the particle is set to the mean energy of the beam plus a sampling from a Gaussian distribution, with the standard deviation stored in the **Beam_energy_spread** variable.

Using the function given, we calculate the current voxel index of the particle within the grid (line 185).

To simulate energy straggling and multiple Coulomb scattering, we created a loop that only ends when the energy of the particle reaches 0. Inside this loop, we considered all the interactions that the particle can undergo. First, however, we added a condition to verify that the particle is still inside the grid; if not, the loop stops. To compute the stopping power, which refers to the current kinetic energy of the particle, we interpolated the given array of stopping power values to get the matching stopping power of our energy.

As explained in II.A of the paper, the step length is defined as the distance between two voxel boundaries (in our case, it is saved in the variable **step_length**) when there is no discrete interaction. However, if there is a discrete interaction, one step is taken until this interaction point. To compute the actual step length of the particle, we created the function **actual_step_length**, which calculates the minimum between **step_length** and the distance to the discrete interaction point, given by the function **compute_distance_to_interface**.

Once we have calculated the actual step, we can calculate the energy loss given by the stopping power within the path of the actual step length, using the formula: $\Delta E = SP \times \rho \times \Delta Z$.

As explained in II.E of the paper, we must also account for the energy fluctuations of the primary protons during the production of secondary electrons in ionization interactions, as these fluctuations influence the steepness of the depth dose distribution beyond the Bragg peak. Energy straggling refers to the random variation in energy loss and is modeled by Landau's theory for thin absorbers, or by

Bohr's theory (Gaussian distribution) for thick absorbers. The straggling function can be represented as a Gaussian with a mean equal to the mean kinetic energy of the proton at the end of the step, and a variance given by equation 16 in Fippel et al.

After calculating β, γ, and T_{max}, we applied equation 16 from Fippel et al. to compute the variance and subsequently updated the energy loss to include the contribution from straggling.

As explained in II.E of Fippel et al., we can approximate the small-angle deflections due to Coulomb elastic interactions with atomic nuclei using a Gaussian distribution, with the variance given by equation 17. After calculating the proton momentum, we applied equation 17 to compute the standard deviation.

In the final part of the loop, we updated all the important features of the particle:

- Subtracted the particle's energy by the total energy loss (where we will also add the energy lost due to nuclear interactions).
- Updated the particle's position by multiplying the actual advance step by the direction.
- Calculated the deflection angle as a Gaussian distribution with a standard deviation equal to θ_0 (which will also be updated with nuclear interactions).
- Calculated the new direction of the particle using the **cdirect** function and the deflection angle calculated in the previous step.
- Updated the **scoring_grid** by adding the energy loss of the particle due to the current interaction in the corresponding voxel.

Lastly, we check the remaining energy of the particle. As mentioned in the paper, if the particle's energy is less than 0.5, it is absorbed locally. In this case, we add the remaining energy of the particle to the corresponding voxel of the **scoring_grid** and update the particle's energy to 0.

I.B Nuclear reactions

To compute the nuclear interactions, we decided to modify the loop in the **CSDA()** function by adding them before updating all the important features of the particle. We computed the distance to the next nuclear interaction by creating the function **dz_nucl**, which applies the formula: $\Delta z_{tot} = -\lambda_{tot} * \ln(\xi)$), where λ_{tot} is the total mean path length (inverse of the sum of all cross sections). These cross sections are given as input to the **dz_nucl** function and are calculated by other functions: **macroscopic_cross_section_pp(energy)**, **macroscopic_cross_section_pO_elastic(energy)**, and **macroscopic_cross_section_pO_inelastic(energy),** where the functions are taken from the Fippel paper.

Each of these functions computes the cross section for the three types of nuclear interactions, after first checking that the proton energy is within the valid range for the equation. Since a nuclear interaction only occurs if the distance calculated by the **dz_nucl** function is smaller than the **actual_step**, we added a condition to verify this (in line 215), ensuring that the calculation of nuclear interactions only takes place if this condition is met. Under this condition, we update the actual step and compute the type of nuclear interaction using the function **sample_interaction_type**, which samples the type of interaction based on a random number from a uniform distribution.

Once the type of interaction is saved inside the type variable, we use if conditions to calculate the three different interactions:

1. **Line 218:** Calculation of the elastic proton-proton interaction. We calculate the cosine of the deflection angle of the incident particle in the center-of-mass system. Using this angle and the energy of the incident particle, we compute the energy transferred to the other particle (Line 222). Next, using the function **CM_to_labframe**, we calculate the cosine of the deflection angle of the incident particle in the laboratory system. We update the total energy lost by the particle by adding the energy transferred during this interaction, and we also update the deflection angle by including the change caused by this interaction. We calculate the deflection angle of the target particle, as well as its position and direction, so that we can add the new proton to the structure and compute all its future interactions using the **CSDA** function with the appropriate input data.
2. **Line 238:** Calculation of the elastic proton-oxygen interactions. We create the function **To_sample** (which applies equations 24 and 25 from Fippel et al.). This function first computes the mean energy transferred to the oxygen particle, and using this value, it samples the actual energy transferred by applying equation 25. We also create the function **deflectionAngle_po_e** to compute the cosine of the deflection angle in the center-of-mass system for both the incident and target particles. Using the same function for proton-proton interactions, **CM_to_labframe**, we then calculate the respective cosines of the deflection angles in the laboratory system. Finally, we update the energy loss and the deflection angle of the incident particle.
3. **Line 250:** Calculation of the inelastic proton-oxygen interactions. Secondary particles are emitted only if the energy of the system is greater than 3 MeV. We create a loop that verifies this condition. After sampling the secondary particle's energy from a uniform distribution between 3 MeV and the energy of the system, we also sample the type of the secondary particle and store it in the variable **type_p**. Since we only need to add the secondary particle to the simulation queue if it is a secondary proton, we use an if condition to check the particle type. In the case where the secondary particle is a proton, we calculate its position and direction and then add this new particle to the grid and simulation queue using the **CSDA** function, as done for the proton-proton interaction. We update the system's energy for the next iteration and also update the energy loss of the incident proton.

I.C Testing

Calculating the integral depth dose, considering nuclear interactions, along with the phantom's density and the given spot size. The results were performed considering 5000 initial particles.

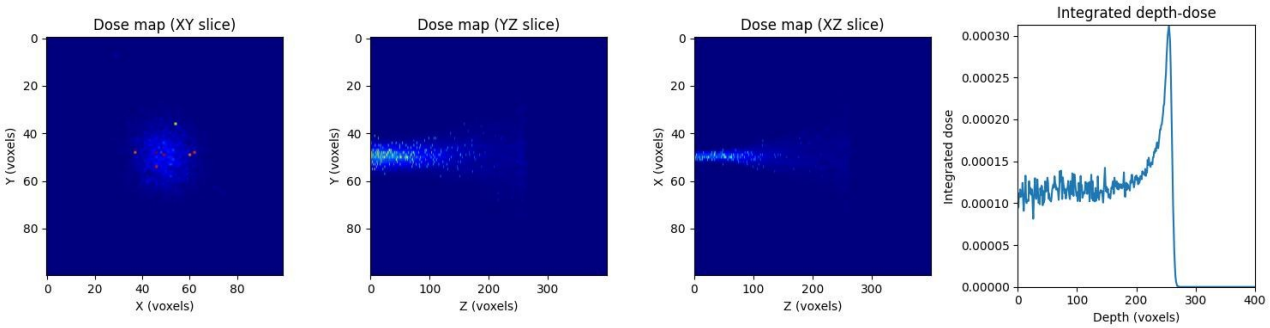


Figure 1: results of the integral depth dose, computed with 5000 initial particles

We attempted to calculate the results using 10,000 particles; however, we did not observe a significant change in the beam distribution

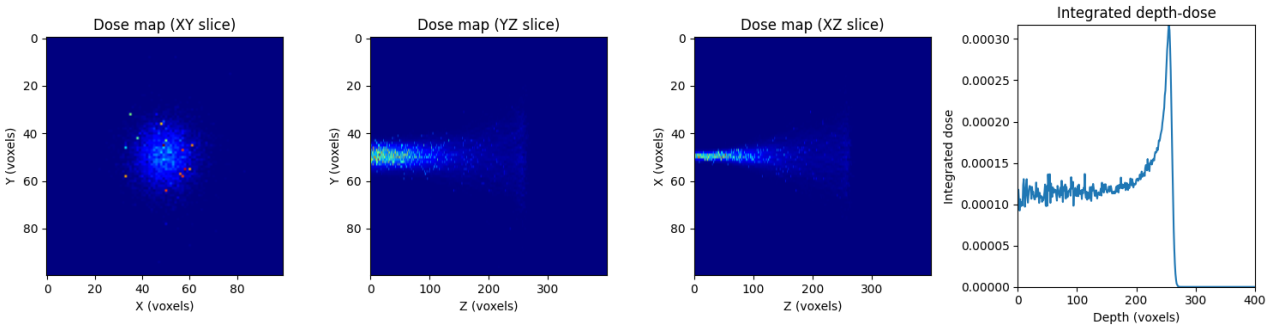


Figure 2 : results of the integral depth dose, computed with 10000 initial particles

We can compare the results with the reference values and observe that the beam is slightly more spread out along the transverse axis and that our initial dose is slightly higher. However, the Bragg peak occurs at the same depth, and the dose delivered to the phantom, as it varies, is also very similar. Less noise is noticed on the reference curve, probably because a lot more particles were simulated.

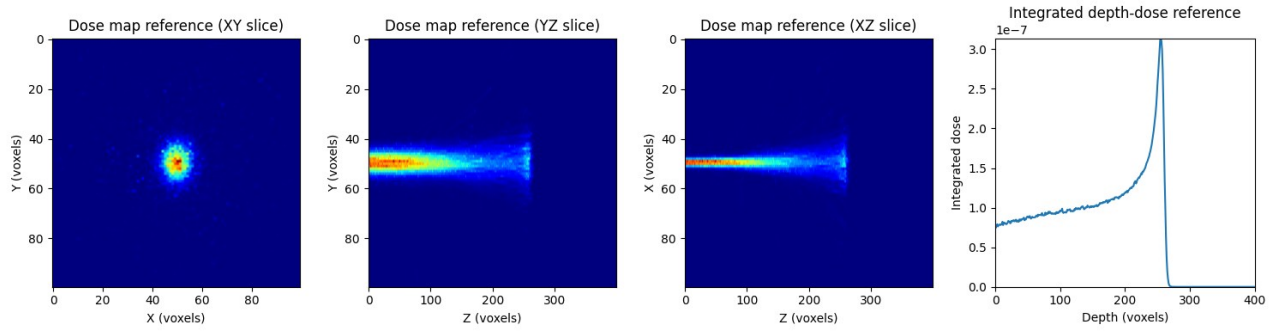


Figure 3: reference integral depth dose

By setting the density of the half left to 2g/cm^3, we got the following results:

Increasing the density of the medium had the result that the particles did not reach the central XY slice (hence the empty plot at the left). The depth of the curve is divided by two. The result would be the same if the total density map was set to 2g/cm^3

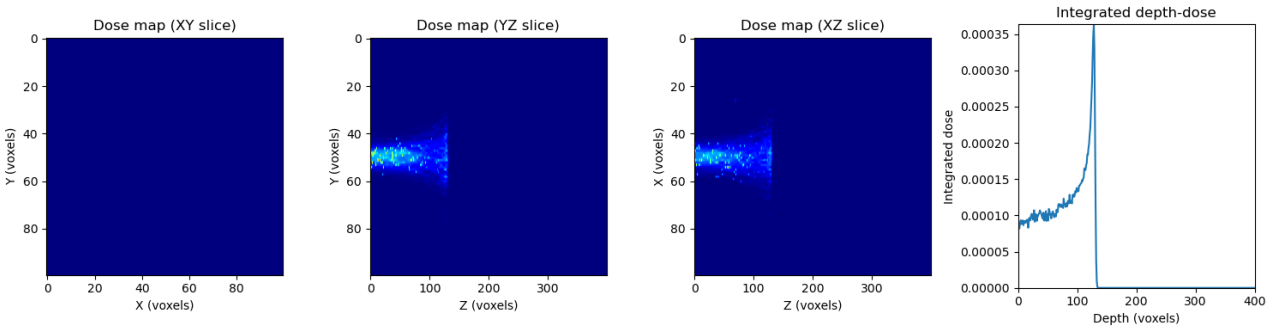


Fig 4: density doubled

II.A Implementation of additional features

We've put the data for compact bone from ICRU in the folder 'data' and added this to the SP_database variable. To simulate a homogeneous phantom, the density of the phantom was set to 1.85 g/cm^3 (taken from ICRU). Before interpolating the stopping power, the density of the current voxel is checked to match it with the right stopping power (water or bone). To convert the dose to water, deposited energy is multiplied by the stopping power ratio. The relative electron density was taken at 1.7 for compact bone (ICRU). The higher stopping power does not have a big effect, as it is relatively similar to water.

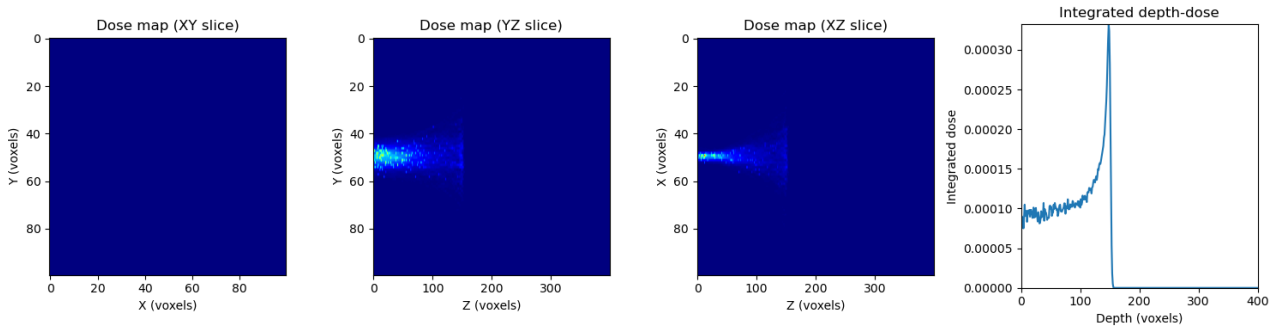


Fig 5: Homogeneous phantom of compact bone

For the heterogenous phantom, we see that the depth of the curve is between the homogeneous compact bone curve and the one for water, as expected. We see larger fluctuations in the dose, as the deposited energy can vary due to the different density and stopping power.

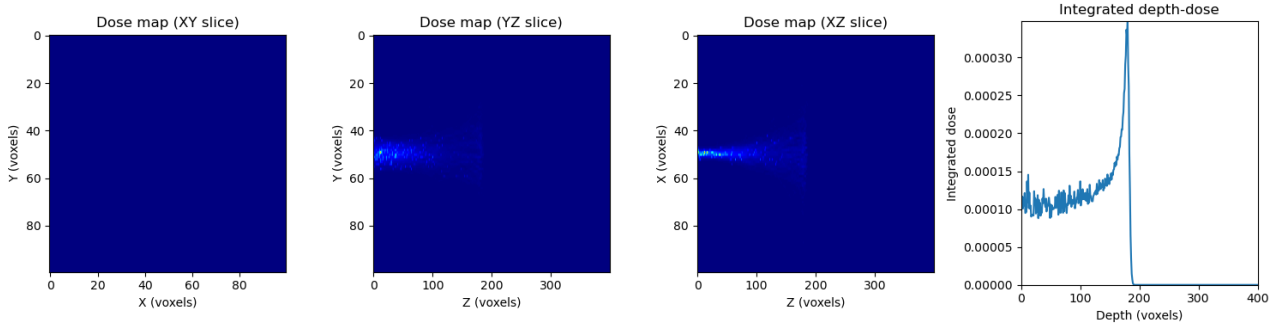
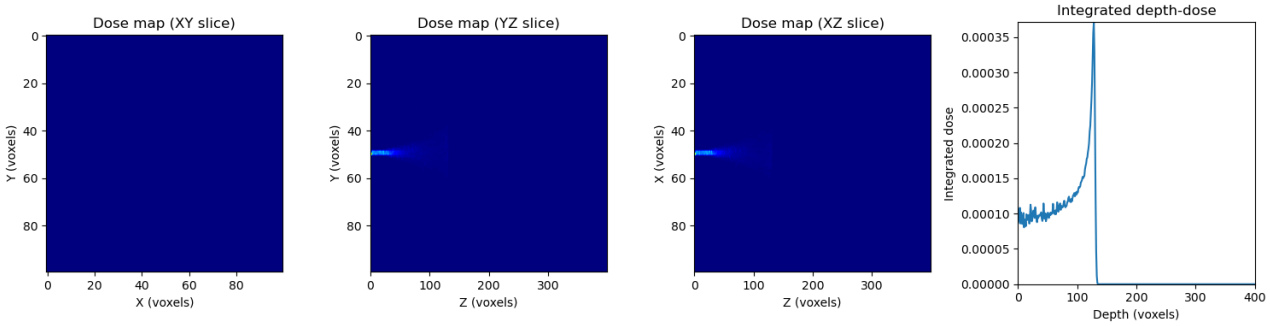


Fig 6: Heterogeneous phantom of compact bone and water

II.B central axis approximation

We can set the value of 'central_axis' to True to simulate a beam with no initial spread in position. By comparing the plot, we see that the spread of the beam is smaller (as expected) and that the integrated depth dose curve is the same. This is logical, as we integrate along the axis, making it independent of the initial spread


Dose map (XY slice)
Y (voxels)
X (voxels)
Dose map (YZ slice)
Y (voxels)
Z (voxels)
Dose map (XZ slice)
X (voxels)
Z (voxels)
Integrated depth-dose
Integrated dose
Depth (voxels)